Traitement d'automate fini

PROGRAMME A DEVELOPPER

Votre programme se déroule en plusieurs étapes :

- lecture d'un automate que vous coderez préalablement dans un fichier, puis mise en mémoire et affichage sur l'écran ;
- affichage des informations : l'automate est-il déterministe ? standard ? complet ?
- si l'automate n'est pas standard, standardisation à la demande de l'utilisateur
- si l'automate n'est pas déterministe complet, obtention de l'automate déterministe et complet équivalent
- calcul de l'automate minimal équivalent ;
- test de reconnaissance de mots ;
- création d'un automate reconnaissant le langage complémentaire

Lecture d'un automate dans un fichier

Le 16 mars nous vous donnerons des automates de test. Ceux-ci devront être transcrits par vos soins dans des fichiers « .txt ».

Le choix de l'automate sera effectué par l'utilisateur pendant l'exécution de votre programme, et ce choix doit pouvoir se faire de façon très simple : par exemple, pour choisir l'automate de test n° 8, il doit suffire de taper « 8 » en réponse à la question « quel automate voulez-vous utiliser ?».

Tant que vous n'avez pas encore reçu les automates de test (qui vous seront fournis sous forme graphique !), vous êtes libres de travailler sur n'importe quel automate de votre invention, en le codant dans un fichier .txt selon le mode que vous verrez plus loin dans ce document.

Affichage de l'automate

Affichage de l'automate sauvegardé en mémoire, en indiquant explicitement :

- l'état initial ou les états initiaux ;
- le ou les états terminaux ;
- la table des transitions.

Par exemple :

		a	b	c
E	0	0, 1	0, 4	0
	1	--	--	2
	2	--	3	--
S	3	3	--	--
S	4	--	--	2

(Attention, la lisibilité de la table est capitale : les colonnes doivent être bien alignées, indépendamment de la longueur du contenu de la cellule ; les lignes verticales et horizontales séparant les cellules sont optionnelles)

Les étapes de lecture et d'affichage peuvent être schématisées par le pseudo-code suivant :

```
AF ← lire_automate_sur_fichier(nom_fichier)
afficher_automate(AF)
```

Standardisation si demandé

```
SI non_standard(AF)
Alors
     SI l'utilisateur veut le standardiser
          SFA ← standardisation(AF)
```

Déterminisation et complétion

Soit l'automate « AF » obtenu (et affiché) à l'étape précédente. Le traitement se déroule selon le pseudo-code suivant :

```
          SI est_un_automate_deterministe(AF)
          ALORS
               SI est_un_automate_complet(AF)
               ALORS
                    AFDC←AF
               SINON
                    AFDC←completion(AF)
               FINSI
          SINON
               AFDC←determinisation_et_completion_automate(AF)
          FINSI
     FINSI
     afficher_automate_deterministe_complet(AFDC)
```

où :

```
est_un_automate_deterministe(AF)
```

Vérifier si l'automate AF est déterministe ou non.

Le résultat du test est affiché. Si l'automate est non déterministe, votre programme doit en afficher les raisons.

`est_un_automate_complet(AF)`

Vérifier si l'automate déterministe AF est complet.

Le résultat du test est affiché. Si l'automate n'est pas complet, votre programme doit en afficher les raisons.

`completion(AF)`

Construction de l'automate déterministe et complet à partir de l'automate synchrone et déterministe non complet AF.

`determinisation_et_completion_automate(AF)`

Construction de l'automate déterministe et complet à partir de l'automate non déterministe AF.

Il s'agit ici de mettre en œuvre le processus de déterminisation vu en cours et TD.

`afficher_automate_deterministe_complet(AFDC)`

Affichage de l'automate, sous un format équivalent à ce qui a été utilisé pour l'automate initial.

En plus de l'affichage de l'automate déterministe complet (AFDC), votre programme doit explicitement indiquer à quels états de l'automate non déterministe en entrée du traitement (AF) correspond chacun des états de l'automate déterministe complet (AFDC).
Les états de l'automate déterminisé doivent être notés de telle façon que leur composition en états de l'automate d'origine soit claire : par exemple, « 123 » pour un état composé de 1, de 2 et de 3. Si les numéros d'états dépassent 9, attention de faire la différence entre {1,2,3} et {12,3}, que l'on notera respectivement, par exemple, « 1.2.3 » et « 12.3 » avec un séparateur de votre choix.

Attention :

Les différents tests mentionnés dans le pseudo-code sont impératifs. Par exemple, la déterminisation d'un automate ne doit être lancée que si l'automate a été explicitement identifié comme n'étant pas déjà déterministe. La déterminisation d'un automate qui l'est déjà sera considéré comme une erreur.

Minimisation

Il n'y a pas de test à effectuer. On minimise l'automate déterministe complet obtenu précédemment et on affiche le résultat. S'il s'avère que l'automate à minimiser était déjà minimal, votre programme doit afficher un message correspondant.

Pseudo-code :

```
AFDCM← minimisation(AFDC)
afficher_automate_minimal(AFDCM)
```

où :

`minimisation ( AFDC )`

Construction de l'automate déterministe, complet et minimal (AFDCM) à partir de l'automate, déterministe et complet (AFDC).

Votre programme doit afficher les partitions successives (numérotées !), ainsi que les transitions exprimées en termes de parties, tout au long du processus de minimisation. Faites attention à ce que cet affichage soit facilement lisible.

`afficher_automate_minimal ( AFDCM )`

Affichage de l'automate sous une forme similaire aux précédentes.

Votre programme doit notamment afficher de façon explicite à quels états de l'automate déterministe complet (AFDC) chaque état de l'automate minimal (AFDCM) correspond.

Il se peut qu'il ne soit pas pratique de nommer les états de l'automate minimal en évoquant les états de l'automate que vous minimisez : par exemple, si « 2.3 » et « 0.4.5 » sont les noms d'états d'un AFDC, il ne serait pas facile de lire le résultat si, par exemple, un des états est nommé « 2.3, 0.4.5, 1 ». Donc vous avez le droit de renommer les états de l'automate minimal. Mais alors, vous devez impérativement afficher une table de correspondance des états de l'automate que vous avez minimisé et l'automate minimal.

Reconnaissance de mots

Votre programme procède à l'analyse de mots fournis au clavier par l'utilisateur.

Votre programme doit impérativement permettre de saisir plusieurs mots et lancer la reconnaissance sur chacun d'entre eux.

Pseudo-code

```
lire_mot ( mot )
TANT QUE mot ≠ « fin » FAIRE
      reconnaitre_mot ( mot , A )
      lire_mot ( mot )
REFAIRE
```

où :

`lire_mot ( mot )`

Récupération d'une chaîne de caractères donnée par l'utilisateur au clavier de l'ordinateur.

Attention :

-Le mot est lu en entier sur une ligne avant vérification. Il ne doit pas y avoir une lecture / vérification caractère par caractère.

-A vous de déterminer le mot correspondant à « fin de lecture des mots ». Le pseudo-code propose la chaîne « fin », mais vous pouvez choisir ce que vous voulez.

`reconnaitre_mot ( mot , A )`

Utilisation de l'automate déterministe complet A pour vérifier si un mot appartient ou non au langage.

En résultat : « oui » ou « non ».

Langage complémentaire

Pseudo-code :

```
AComp ← automate_complementaire ( A )
afficher_automate ( AComp )
```

où :

`automate_complementaire ( A )`

Construction d'un automate reconnaissant le langage complémentaire.

Le paramètre d'entrée A peut être, à votre choix, l'AFDC ou l'AFDCM obtenu précédemment. Votre programme doit indiquer à partir de quel automate le complémentaire est obtenu.

Boucle de traitement de plusieurs automates

Il est impératif de mettre tous les traitements identifiés ci-dessus dans une boucle générale permettant de traiter plusieurs automates AF sans relancer votre programme.

ENVIRONNEMENT DE PROGRAMMATION

Vous pouvez utiliser les langages C ou Python.

AUTOMATES A PRENDRE EN COMPTE

Les tests seront effectués sur des automates :

- ayant pour alphabet les premières lettres de l'alphabet latin au nombre indiqué dans le fichier .txt de l'automate, sans rupture de séquence:

  par exemple, un automate dont l'alphabet contient 3 symboles utilisera les lettres 'a', 'b' et 'c' ;

- dont les états sont numérotés à partir de '0' :

  par exemple, un automate contenant 5 états contiendra les états numérotés de 0 à 4, sans rupture de séquence.

Il n'y a pas de limite concernant le nombre d'états des automates de test.

Le fichier de données représentant l'automate lu par votre programme peut avoir la syntaxe suivante (mais vous pouvez opter pour une syntaxe un peu différente) :

Ligne 1 : nombre de symboles dans l'alphabet de l'automate.

Ligne 2 : nombre d'états.

Ligne 3 : nombre d'états initiaux, suivi de leurs numéros.

Ligne 4 : nombre d'états terminaux, suivi de leurs numéros.

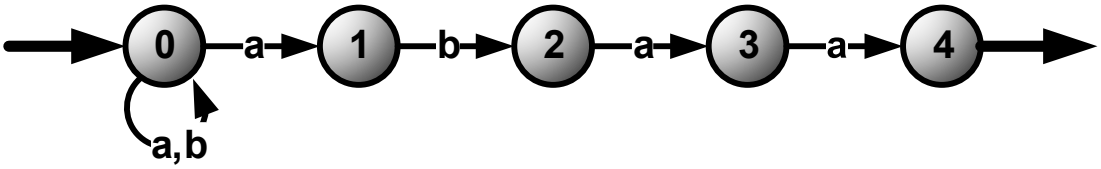
Ligne 5 : nombre de transitions.

Lignes 6 et suivantes : transitions sous la forme

<état de départ><symbole><état d'arrivée>

Par exemple :

Automate :



2 symboles dans l'alphabet
A={a,b}
5 états
Q={0,1,2,3,4}
1 état initial
I={0}
1 état terminal
T={4}
6 transitions

Fichier (par exemple):

```
2
5
1 0
1 4
6
0a0
0b0
0a1
1b2
2a3
3a4
```

DEROULEMENT DU PROJET

Organisation du travail

Travail par équipes :

- il faut au plus 8 équipes par groupe de TD.
- il faut au moins trois étudiants dans une équipe.
- il faut au maximum cinq étudiants dans une équipe.
- **Date limite pour la constitution des équipes : dimanche 8 mars à 23h59.** Les délégués doivent envoyer aux enseignants de TD la liste de constitution des équipes.
- Problèmes éventuels: S'il y a problème au niveau d'une équipe (un élève qui ne participe pas au travail des autres, un élève malade, etc...), vous êtes tenus d'en informer votre enseignant immédiatement. Aucune modification d'équipe sans l'aval direct de l'enseignant ne sera acceptée.

Les automates de test qui seront utilisés pour la soutenance vous seront communiqués le **16 mars**.

Remise de votre travail

Toutes les équipes devront remettre leur travail sur Moodle pas plus tard que le **29 mars à minuit**.

Le contenu du rendu :

- Code source : Tout fichier code que vous avez tapé vous-même à l'exclusion de tout autre fichier (**donc aucun fichier produit par le logiciel durant la compilation ou exécution**), bien commenté.
- Tous les fichiers .txt des automates de test (oui, malgré le fait que ce sont vos enseignants qui vous ont fourni les automates de test), **dans le même répertoire que le code**.
- Les traces d'exécution sous forme d'autant de fichiers .txt qu'il y a d'automates. Vous devrez exécuter votre programme sur l'intégralité des automates de test mentionnés ci-dessus et fournir les traces d'exécution correspondantes.

Un automate non testé, ou pour lequel les traces d'exécution ne seront pas fournies, sera considéré comme un graphe sur lequel votre programme ne fonctionne pas correctement.

Soutenance

Durée : 30 minutes par équipe.

- Il n'y aura pas d'ordinateur mis à votre disposition. Vous devez impérativement venir avec le vôtre en vous assurant préalablement que celui-ci marche et compile.
- 30 minutes par équipe. Vous aurez 10 minutes au début pour faire une démonstration de votre programme, puis 20 minutes de questions/réponses. Inutile donc de préparer un diaporama ppt de présentation.
- La note de la soutenance est collective. Durant la soutenance, votre enseignant pourra poser n'importe quelle question à n'importe quel membre de l'équipe, qu'il participe à l'élaboration de la partie du programme concerné ou non. Tous les membres de l'équipe sont donc responsables de s'assurer que tous les autres membres ont le même degré de compréhension du projet.

Les soutenances ont une durée limitée. Votre enseignant à un planning très serré. Il ne pourra pas continuer les soutenances après le créneau horaire prévu. Il vous est vivement conseillé d'être vraiment prêt à l'heure du début de votre soutenance, ce qui implique (au cas où une soutenance physique a lieu) :

– attendre « à la porte de la salle » et « entrer » dès que c'est à votre tour ;

– avoir préparé votre ordinateur, y avoir inclus tous vos programmes et fichiers contenant les automates ;

– avoir vérifié le chargement de la batterie de l'ordinateur ;

– l'avoir démarré et mis en mode veille ;

– avoir un ordinateur de secours sur lequel vous avez aussi vérifié le bon fonctionnement de votre programme, au cas où...

Tous les ans, il y a plein d'étudiants qui pensent ne jamais avoir de problème. Tous les ans, il y en a qui en ont. Résultat : ils sont stressés et ont moins de temps pour leur soutenance. Dommage…